R Programming Language Evaluation Report

I. Suitability for the Problem

R is suitable for implementing the core functionality of the application because it supports numerical computation, conditional logic, functions, and useful data structures such as hash maps which are modeled using lists in R. In the implementation, account information is stored using nested lists, while the jsonlite packages handle persistent storage through JSON files. These features allow the application to manage account balances, perform currency conversions, and compute projected daily balances accounting the interest without requiring a complex external infrastructure. However, R's primary strengths lie in statistical computing and data analysis rather than general-purpose application development, which may make it less suitable if the system eventually requires extensive backend services or concurrent transaction, raising questions to its scalability.

Code Block 1. Jsonlite's fromJSON function reading a JSON file

```
16  #reads and loads the account records from the json file and returns it
    1 call
17  load_accounts <- function() {
18      tryCatch(expr = {
19          accounts <- jsonlite::fromJSON(txt = "registered_accounts.json",
20          simplifyVector = FALSE)
```

Code Block 2. The contents of the JSON file are returned and assigned to accounts

```
19  #registered accounts will be stored here upon runtime
20  accounts <- load_accounts()
```

The variable "accounts" uses a nested lists (hash maps) where the key is the account name and value is another list that stores account details such as balance and other more fields if the programmer wishes to add more account details in the future.

Code Block 3. Exchange Rates Data Structure

```
 9  #source: php (last updated: oct 13, 2026)
10  exchange_rates <- c(
11      "Philippine Peso (PHP)" = 1,
12      "United States Dollar (USD)" = 0.016,
13      "Japanese Yen (JPY)" = 2.52,
14      "British Pound Sterling (GBP)" = 0.012,
15      "Euro (EUR)" = 0.014,
16      "Chinese Yuan Renminni (CNY)" = 0.11
17  )
```

Exchange Rates are stored in a named numeric vector resembling a hash map, with currency names serving as keys and their corresponding exchange rates as values.

II. Expressiveness and Readability

R offers expressive syntax for representing data, particularly through vectors and named lists, membership checks, and loops. However, some of its syntax for input validation, data-access conventions, and assignment statement may be less intuitive to programmers unfamiliar with the language. Although R enables concise implementations, conciseness doesn't always translate to readability. Therefore, the readability of an R program depends on the programmer's familiarity with the language, it is not like other languages where its syntax is highly recognizable from other languages.

Code Block 4. How valid integer inputs are checked

```
33      if (!grepl(pattern = "^-?[0-9]+$", choice)
```

Code Block 5. Membership checks

```
85      #checks if account name already exists in the registered accounts
86      if(account_name %in% names(accounts)){
```

Code Block 6. For-Loop

```
57  for (i in 2:n){
58      inv <- convert_currency(amount = 1, from = i, to = 1, exchange_rates)
59      cat(sprintf(fmt = "%-30s| %-10.3f| %-10.3f\n", names(exchange_rates)[i],
60      exchange_rates[[i]], inv))
61  }
```

Code Block 7. Data-access and update

```
146     #update balance
147     accounts[[account_name]]$balance <- current_balance + deposit_amt
```

III. Reliability and Safety

R provides mechanisms for developing reliable software, such as tryCatch() for error handling and conditional statements for implementing validation rules. R also mitigates the risk of committing memory-management errors via automatic memory management and garbage collection. However, these features do not guarantee that a program is safe and reliable, as programmers must still implement appropriate and stricter safeguards in handling invalid inputs and type mismatch as R is dynamically typed and you don't want

your balance to be storing a string, negative deposit and withdrawals, duplicate account names, and withdrawals exceeding the available account balance.

Code Block 7. Try Catch (Handling Errors during runtime)

```r
#updates the account database everytime a change in accounts occur
4 calls
update_accounts <- function(accounts) {
    tryCatch(expr = {
        jsonlite::write_json(accounts, path = "registered_accounts.json",
        auto_unbox=TRUE, pretty=TRUE)
        TRUE
    }, error = function(e) {
        cat("Failed to save account: ", conditionMessage(e), "\n")
        FALSE
    })
}
```

IV. Tooling and Ecosystem

R provides a range of tools and libraries that support application management. In this project, Visual Studio Code serves as the development environment, with the R extension to integrate R development features into the workspace. The R interpreter gives you the option to run only a specific selected portion of a larger program independently, facilitating testing and debugging without having to run the entire program each time. The jsonlite packages simplify JSON serialization and deserialization, allowing account data to be stored persistently and be loaded back upon runtime. Furthermore, R's package management system makes external libraries easily accessible through commands such as install.packages(). R also provides a built-in help system and extensive documentation for its function and packages, making it easier for developers to understand features and troubleshoot issues.

V. Performance and Scalability

In terms of performance, R satisfies every core functional requirement of the application, including currency conversions, balance updates, and interest calculations. Since the application processes a relatively small amount of account data, R is sufficient for its current requirements. However, as the number of users and account details grow, performance may be affected by inefficient code and growing memory requirements. If the application must handle multiple users simultaneously, concurrency will become necessary, although implementing it introduces additional complexity and a more thorough handling of data to maintain consistency.

VI. Comparison and Reflection

Compared with python, R offers a similar level of convenience for many basic programming operations, including membership checks, loops, user input, and file I/O. For example, R's "%in%" operator serves a similar purpose to python's "in", making both languages very straightforward to use for checking whether an account already exists in the registered accounts database. The file I/O are also very similar in nature, they both don't require a programmer to specify the format in which how should the program should serialize or deserialize the file and object data. Both languages are also dynamically typed offering flexibility and speed, but it significantly increases the risk of runtime errors, this forces programmers to implement tighter guardrails to mitigate the risks of costlier problems runtime bugs poses, such as database corruption. Furthermore, both languages offers significant abstraction relative to lower-level languages, taking away the headaches found in variable declaration and memory management. For this project, R provides sufficient functionality to implement the required banking operations, persistent data, and currency exchange operations. Python might be a more a suitable choice, particularly if the application were expanded to include more extensive integration and additional application features. Ultimately, both languages can fulfill the project's requirement, the choice depends on the programmer's familiarity with the language as the differences mainly lie on the syntax.